

How F2Z performs the forest GKR: the merged, lazy, bit-affine grand-product forest

The `f2z-pcs` repository

2026-07-27

Abstract

F2Z’s integer-MLE opening rides on a grand-product argument: per column c of the committed bit matrix, the prover must certify $\alpha^{u_c} = \prod_i \Lambda[c][i]$ for the *bit-affine* leaves $\Lambda[c][i] = 1 + M[c][i] \cdot \tau(i)$. This is the prover’s dominant phase (81–96 % of prove time across $n = 16$ –32), and this note explains how it is actually performed, as implemented in `src/merged_forest.rs` and the eq-factored sumcheck driver it runs on. Three ideas carry the design. (i) *Merging*: the 2^s product trees are one multilinear family with the tree index as extra MLE variables, so one GKR run binds every root and each layer’s payload is $O(\ell + s)$ field elements instead of $O(2^s)$. (ii) *The two-phase eq-factored layer sumcheck*: phase A binds the in-tree variables with the trees as eq-weighted groups sharing one reduction point; phase B binds the tree-index variables over the per-tree finals, which never enter the transcript. The driver materialises no `eq` tables, accumulates round coefficients in unreduced 256-bit form, and gets the fourth Lagrange node for free from a characteristic-2 affine-flat identity. (iii) *Laziness*: the leaf layer and the bottom product levels are never built — the bottom layers run branchless case-LUT sumcheck rounds straight off the packed committed bits against shared τ -product tables, one intermediate level is regenerated just in time (with its first round message fused into the generation pass), and the stored chain tops out a quarter (default schedule) or an eighth (18) of the way up the tree, which is what caps prover memory at $\approx 2^n \cdot 4$ B resp. $\approx 2^n \cdot 2$ B. Every lazy path is an exact characteristic-2 rewriting of the eager forest: the transcripts are byte-identical, and tests pin this in both schedules. The note closes with the exit-claim discharge into the ring-switch/Ligerito opening, the batched and RLC-family variants of the forest, and the measured cost tables.

Contents

1	The job of the forest	2
2	One forest, one transcript: the merged statement	3
3	The layer protocol	4
4	The eq-factored driver	5
5	Leaves from bits: the lazy schedules	6
5.1	What the prover holds	6
5.2	The level ladder	7
5.3	Inside the case-LUT rounds	7
5.4	Why byte-identity holds, and how it is pinned	8

6	Memory and cost	9
7	The exit claim and its discharge	10
8	Variants: the batched forest and the RLC families	10
9	What to trust, and what this note is not	11

1 The job of the forest

F2Z¹ proves an *integer* multilinear-extension evaluation $\widetilde{\text{INT}}(D)(r) = y \in \mathbb{F}_q$ over data D whose *bits* are committed with a characteristic-2 commitment. The instance shape is (t, s, W) (`IntEvalParams`): D is a $2^t \times 2^s$ matrix of W -bit cells, its bits arranged as the single-bit matrix

$$M[c][i] \in \{0, 1\}, \quad i = (b \ll \log_2 W) | j \in \{0, 1\}^d, \quad d = t + \log_2 W,$$

with b the row, c the column, j the bit position inside the cell. The t row variables are folded with integer weights, the s column variables are read off in the clear. We write $n = t + s$ for the benchmark-scale parameter (the deployed shapes have $W = 1$, so 2^n is the committed bit count) and $\mathbb{K} = \text{GF}(2^{128})$ for the binding field, with $\alpha \in \mathbb{K}^\times$ a generator.

The claim the forest certifies. An integer fold cannot be read through the $\mathbb{F}_2$ commitment (parities destroy magnitudes), so F2Z carries it in the exponent of $\mathbb{K}$: for one chunk l of the mod- q opening, with public c_w -bit chunk weights $W_b^{(l)}$ (limbs of the $\mathbb{F}_q$ row-eq weights, $c_w = 127 - t - W$), the prover sends the per-column integer folds

$$u_c^{(l)} = \sum_b W_b^{(l)} \cdot \text{INT}(D(b, c)) < 2^{c_w+t+W} = 2^{127},$$

and must certify them against the commitment. The certificate is a grand product: each factor is *bit-affine* in one committed bit,

$$\Lambda^{(l)}[c][i] = 1 + M[c][i] \cdot \tau^{(l)}(i), \quad \tau^{(l)}(i) = \alpha^{W_b^{(l)} 2^j} - 1 \quad (i = (b \ll \log_2 W) | j), \quad (1)$$

so that $\prod_i \Lambda^{(l)}[c][i] = \alpha^{u_c^{(l)}}$. (In $\mathbb{K}$, $-1 = +1$; we keep the -1 form for readability.) The forest's output is a reduction of all 2^s product claims to *one* evaluation claim on the leaf MLE $\tilde{\Lambda}$, which §7 converts into a $\mathbb{K}$ -linear functional of the committed bits and hands to the ring-switch + Ligerito opener. Range checks on the sent u 's and the generator property of α make the exponent map injective on the values that occur, so the products bind the integers; that chain of checks is the verifier note's §§5–7, and we do not repeat it here.

Why performance of this phase is the whole game. Time is forest-dominated: the forest+presum phase is 81 % of prove at $n = 16$, rising to 92–96 % at $n \geq 26$ (§6). Proof *bytes*, by contrast, are dominated by the Ligerito opening — the forest side stays at 8–51 KiB through $n = 28$. So the forest is engineered for prover time and memory, and its transcript is engineered to stay small; both axes are visible in the design below.

¹Repository `f2z-pcs`, a standalone extraction of the `f2-int` pipeline from `zinc-plus` (branch `f2-int-unified-ligerito`); the construction is due to Lev Soukhanov (the `char2-fieldswitch` note, §9). `docs/DESIGN.md` holds the distilled protocol; `docs/verifier-note/` specifies every verifier check; `docs/rlc-family-note/` develops the experimental claim families. This note is the *prover-side* companion: how the forest is computed, scheduled, and kept small.

Scope. Everything in this note is the code of this repository: the merged forest (`src/merged_forest.rs`), the eq-factored driver (`src/piop/sumcheck/eq_factored.rs`), and the table builders (`src/pcs.rs`). Function names are given so claims can be checked against source; line numbers are avoided (the files drift). The Ligerito opener is out of scope (black box, as in the verifier note).

2 One forest, one transcript: the merged statement

A naive realisation runs one GKR product tree per column and carries, per tree per layer, a pair of child evaluations — $2 \cdot 2^s \cdot d$ $\mathbb{K}$ -elements of proof, ≈ 512 KiB at $n = 26$. The merged forest removes the 2^s factor from the payload by treating the tree index as *more MLE variables*.

The layer family. Index the values of tree c at depth ℓ (the root is depth 0, leaves depth d) by $x \in \{0, 1\}^\ell$, and define one multilinear family per depth,

$$L_\ell(x, c), \quad (x, c) \in \{0, 1\}^\ell \times \{0, 1\}^s,$$

with x occupying the *low* index bits and c the high ones. The product recursion pairs children on the *top* in-tree bit:

$$L_\ell(x, c) = L_{\ell+1}(x, 0, c) \cdot L_{\ell+1}(x, 1, c), \quad (2)$$

i.e. the two children of node x sit at positions x and $x + 2^\ell$ of level $\ell + 1$. This convention is load-bearing for the lazy schedules: every level is stored and consumed as a top-bit split pair of halves (E, O) — $E_c(x) = L_{\ell+1}(x, 0, c)$, $O_c(x) = L_{\ell+1}(x, 1, c)$ — and the leaf halves are exactly “first 2^{d-1} bits, last 2^{d-1} bits” of a column, which is what lets pair products be table lookups (§5). $L_0(\cdot, c) = R_c$ are the roots and $L_d = \Lambda$ the leaves.

Entry: the roots are derived, absorbed, and evaluated — never sent per-tree. The prover’s product-tree build produces the roots R_c as a byproduct; the proof object carries no root field. On the verifier’s side the roots are *recomputed* as $R_c = \alpha^{u_c}$ from the sent folds (fixed-base comb, `FixedBasePow`) — for an honest prover these are the same values, and that equality is precisely the claim being proven. Both sides then perform the same two transcript actions (`drive_grouped` / `verify_merged_forest`):

$$\text{absorb}(0x30 : R_0, \dots, R_{2^s-1}), \quad \zeta \leftarrow \text{squeeze}(\mathbb{K}^s),$$

and each computes the entry claim itself, $C_0 = \tilde{R}(\zeta)$ (`mle_at`, $O(2^s)$ work). The absorb is the binding: a prover whose subsequent messages were generated against different roots faces challenges derived from *these* roots. Note the prover-side integer folds u_c (`fold_values_bits`, a popcount-style pass over the packed rows) are computed independently of the forest and enter the proof stream as the `us` vector; the forest itself never touches them.

Per-layer payload. Each of the d layers contributes one `MergedLayer` to the proof: an optional phase-A sumcheck (ℓ rounds, absent at $\ell = 0$), a phase-B sumcheck (s rounds), and a closing child pair — $O(\ell + s)$ $\mathbb{K}$ -elements. Summed over layers this is $\approx 48 \left(\frac{d(d-1)}{2} + ds \right)$ bytes of round messages plus $32d$ bytes of pairs (degree-3 rounds carry three 16-byte tail values each, §4); at the deployed shapes the *sent folds* $16 \cdot 2^s$ B are the larger forest-side item (e.g. $n = 28$, $s = 11$: 32 KiB of folds vs. ≈ 16 KiB of forest messages, 50.7 KiB forest-side total).

3 The layer protocol

The verifier maintains a running claim $C_\ell = \widetilde{L}_\ell(z_x, z_c)$ starting from $(z_x, z_c) = (\varepsilon, \zeta)$. By (2) and multilinearity, the layer- ℓ claim factors coordinate-wise:

$$\begin{aligned} C_\ell &= \sum_{x \in \{0,1\}^\ell} \sum_{c \in \{0,1\}^s} \text{eq}(x, z_x) \text{eq}(c, z_c) \underbrace{L_{\ell+1}(x, 0, c)}_{E_c(x)} \underbrace{L_{\ell+1}(x, 1, c)}_{O_c(x)} \\ &= \sum_c \text{eq}(c, z_c) \left[\sum_x \text{eq}(x, z_x) E_c(x) O_c(x) \right]. \end{aligned} \quad (3)$$

The bracketed inner sums share the point z_x and differ only in the tree; that shape is exactly one *shared- q multi-group* instance of the eq-factored driver (§4), and the layer is discharged in two phases plus a line step (**drive_grouped**):

1. **Phase A** (in-tree variables; $\ell \geq 1$ only). The 2^s trees enter as groups: group c has the shared eq point $q = z_x$, the per-group scale $\text{eq}(c, z_c)$ (a precomputed $O(2^s)$ table re-built per layer), and the single buffer pair (E_c, O_c) . One driver call (**prove_eq_inner_sumcheck_mixed_pre**) runs ℓ rounds of a degree-3 sumcheck binding $x \rightarrow r_x \in \mathbb{K}^\ell$, and returns the per-tree finals $(\widetilde{E}_c(r_x), \widetilde{O}_c(r_x))$. These finals are *prover-internal*: nothing per-tree is absorbed at any point.
2. **Phase B** (tree-index variables). The residual $\sum_c \text{eq}(c, z_c) \widetilde{E}_c(r_x) \widetilde{O}_c(r_x)$ is itself an eq-weighted inner product over c — the per-tree finals, read as two length- 2^s multilinear tables $E(\cdot), O(\cdot)$, feed one more (tiny, dense) driver call with $q = z_c$, binding $c \rightarrow r_c \in \mathbb{K}^s$ and closing on the child pair

$$(p, q) = (\widetilde{L}_{\ell+1}(r_x, 0, r_c), \widetilde{L}_{\ell+1}(r_x, 1, r_c)).$$

3. **Line step. absorb(0x32 : p, q), $\lambda_\ell \leftarrow \text{squeeze}(\mathbb{K})$** , and the claim advances along the line through the two children:

$$C_{\ell+1} = p + \lambda_\ell (p + q), \quad z_x \leftarrow r_x \parallel \lambda_\ell, \quad z_c \leftarrow r_c.$$

(The code calls λ_ℓ **mu**; renamed to keep μ for the discharge residual, as in the verifier note.)

Layer 0 degenerates: there are no in-tree variables yet, phase A is skipped (**sc_x = None**), and the pair vectors fed to phase B are simply each root's two children — level 1, two values per tree. After d layers the *exit claim* is

$$e_d = C_d \stackrel{\text{honest}}{=} \widetilde{\Lambda}(z_x, z_c), \quad z = (z_x, z_c) \in \mathbb{K}^{d+s},$$

pinned by the **merged_forest_roundtrips** test against **mle_at(leaves, z)**. The verifier-side chaining checks (phase-A claimed sum $= C_\ell$, the factored-eq linkage $E_A = \text{eq}(r_x, z_x) \cdot \text{sc}_c.\text{claimed_sum}$, the closing check $E_B = \text{eq}(r_c, z_c) \cdot p \cdot q$) are [V5]–[V11] of the verifier note and mirror this structure exactly.

Remark 1 (Why two phases). A single $(\ell + s)$ -variable sumcheck over the merged index would be sound too, but would need the $\text{eq}(c, z_c)$ weights woven into every round over a buffer of size $2^{\ell+s}$. The split lets phase A treat $\text{eq}(c, z_c)$ as a per-group scalar (it seeds the group's prefix scalar, §4, at zero marginal cost), keeps the per-tree buffers independent — which is what the lazy bit-driven rounds and the across-tree parallelism need — and makes phase B negligible (2^s -sized buffers). The transcript cost is one extra sumcheck header per layer.

4 The eq-factored driver

Every phase of every layer runs on one prover, `prove_eq_inner_sumcheck_mixed_pre` (`src/piop/sumcheck/eq_factored.rs`), which proves sums of the shape

$$\sum_x \sum_g \text{eq}(x; q_g) \cdot \text{scale}_g \cdot L_g(x) R_g(x)$$

— an eq-weighted sum of inner products of multilinear pairs — and is **byte-identical** to the generic `MLSumcheck` prover run over the materialised comb $[\text{eq}_1, \dots, L\text{'s}, R\text{'s}]$ with the degree-3 combiner $\sum_g \text{eq}_g L_g R_g$: same round polynomials, same transcript operations (header, tail absorbs, post-draw challenge re-absorb), same proof layout. The generic verifier verifies it unchanged — there is no bespoke verification path to audit.

Never materialise eq. Round j (binding the *low* variable, as the generic prover does) writes each group’s round message as

$$P_j(c) = \sum_g A_{g,j} \text{eq}_1(c; q_g[j-1]) H_{g,j}(c), \quad H_{g,j}(c) = \sum_b V_{g,j}[b] L_g(c, b) R_g(c, b),$$

where $L_g(c, b) = L_g[2b] + c(L_g[2b+1] - L_g[2b])$ is the fold-in-progress buffer at suffix slot b , and the eq weight has been split into three parts, none of them a materialised multiplicand:

- the *prefix scalar* $A_{g,j} = \text{scale}_g \cdot \prod_{i < j-1} \text{eq}_1(\rho_{i+1}; q_g[i])$, one running $\mathbb{K}$ -element per group, updated per round — this is where phase A’s $\text{eq}(c, z_c)$ group scale rides for free;
- the current-variable factor $\text{eq}_1(c; q_g[j-1])$, degree 1 in the round variable;
- the *suffix tensor* $V_{g,j}[b] = \prod_{i \geq j} \text{eq}_1(b_{i-j}; q_g[i])$, precomputed back-to-front for all rounds in $O(2^k)$ total (`suffix_tensors`) — the cost of one eq build, and built *once* for the whole phase-A call, since all trees share $q = z_x$ (asserted: bit-selected groups require the shared point).

H is quadratic and the eq_1 factor linear, so P_j has degree 3, matching the pinned verifier degree.

Coefficient-space accumulation, wide sums, free fourth node. The per-slot inner loop accumulates the *coefficients* (A_0, A_1, A_2) of H in the round variable rather than its node values, so the Lagrange-node multiplies stay out of the hot loop; the single-pair case costs 5 field multiplies per slot (the suffix weight is folded into L : $w \cdot l_0$, $w \cdot l_1$, then three products $(wl_0)r_0$, $(wl_1)r_1$, $\Delta L \cdot \Delta R$). The three products feed `WideMulAcc` accumulators: unreduced 256-bit carry-less sums with *one* polynomial reduction per accumulator per round — exact, because the reduction is $\mathbb{F}_2$ -linear. Round messages are sent as tail evaluations $P_j(1), P_j(2), P_j(3)$ only; the verifier reconstructs $P_j(0)$ from the chained sum (the claimed-sum trick [4]). The fourth node is free in characteristic 2: the evaluation points $\{0, 1, X, X+1\}$ (bit-pattern convention) form an $\mathbb{F}_2$ -affine 2-flat, and over such a flat every polynomial of degree ≤ 2 sums to zero, so $H(X+1) = H(0) + H(1) + H(X)$. The detection is exact at runtime ($1 + 1 = 0$), keeping the driver field-generic. On aarch64 the whole-buffer message and fold passes dispatch to fused NEON kernels (`eqf_single_pair_round` and friends; `F2Z_EQF_NOKERNEL=1` forces the generic loops for A/B diagnosis).

Pass fusion. By default the driver defers each round’s fold and runs it *fused into the next round’s message pass*: one read of the unfolded buffers replaces a fold-write pass plus a message-read pass. Measured $\approx 10\%$ of prove at $n = 26\text{--}28$; byte-identical (same field values in the same transcript order — only the physical pass structure changes); `F2Z_EQF_FUSE=0` restores the eager two-pass path. The same idea appears one level up in the forest’s JIT layers: the *buffer-generation* pass accumulates the round-1 coefficients as it writes (`dense_jit_fused_round1`, forwarded through the driver’s `pre_round1` hook), so a JIT-generated level is written once and first *read* by round 2’s fused fold+message pass (`F2Z_JIT_R1=0` opts out). XOR accumulation is order-free and reduction $\mathbb{F}_2$ -linear, hence generation-order independence of the reduced coefficients — the fusion is again an exact identity.

5 Leaves from bits: the lazy schedules

The eager reference prover (`prove_merged_forest`) materialises the 2^{d+s} leaves and every product level above them. The lazy prover (`prove_merged_forest_lazy`) produces a byte-identical transcript while never materialising the leaf layer or the bottom product levels at all. Everything rests on the observation that at and near the leaves, the forest’s values have so little entropy that they can be *gathered from small shared tables indexed by committed bits* instead of computed and stored.

5.1 What the prover holds

Bits. The commitment hint stores the bit matrix column-lane-packed (64 columns per u64 word). One transposition pass (`extract_column_bit_halves`, blocks of 64×64 bit-transposes) yields, per tree c , the top-bit-split halves of its leaf bits:

$$\text{lbits}_c = M[c][0..2^{d-1}], \quad \text{rbits}_c = M[c][2^{d-1}..2^d],$$

packed 64 per word — a few KiB per tree. These two bit arrays drive *every* lazy layer of tree c ’s bottom.

α -power chains. Per chunk, `chunk_pow2_table` builds $v(i) = \alpha^{W_b^{(l)} 2^j}$ for all $i = (b \ll \log_2 W)|j$: one fixed-base comb for the shared α -squaring chain (`FixedBasePow`, $\approx 8\times$ fewer multiplies than square-and-multiply per exponent), then a per-row squaring chain for the 2^j place values.

Shared value tables. From the chains, four table families are built once per weight set (“ τ set”) and shared by all 2^s trees:

- *leaf τ halves* (`leaf_tau_halves`): $\tau(i) = v(i) + 1$, split like the bits — the affine coefficients of (1);
- the *pair table* (`layer1_pair_table`): $\text{pair}[i] = v(i) \cdot v(i + 2^{d-1})$, the only case of a level- $(d-1)$ value that is not already a table entry;
- the *4-case tables* `te`, `to`: position y of level $d-1$ pairs leaves $(y, y + 2^{d-1})$, so its value is one of $\{1, v(y), v(y + 2^{d-1}), \text{pair}[y]\}$, selected by two bits. `te` tabulates the four cases for the E-half positions $y < 2^{d-2}$, `to` for the O-half positions (offset baked in);
- the *16-case table* $T4$: $T4[(y \ll 4)|(c_E \ll 2)|c_O] = \text{te}[4y + c_E] \cdot \text{to}[4y + c_O]$ — all 16 possible values of the *4-leaf product* at level $d-2$, position y , selected by four committed bits.

Costs are $O(2^d)$ per set — 2^s -fold amortised — and the *T4* build uses an indexed collect with an in-place flatten (a parallel nested `flatten` here measured $\approx 836\times$ slower than the serial arithmetic; upstream fix `e19b0e1`).

5.2 The level ladder

Recall layer ℓ 's phase A consumes level $\ell+1$. The lazy prover sources each level in one of three ways: from the *stored chain* (built bottom-up once, level-major, by `build_levels`), *JIT* (regenerated from bits and tables while its layer runs, then dropped), or *implicitly* (never as $\mathbb{K}$ -values: the layer's first round(s) run case-LUT kernels straight off the bits, §5.3). Two schedules exist, named for the size of the stored chain relative to the 2^d -leaf layer; both are pinned byte-identical to the eager forest.

The L/4 schedule (default; depth ≥ 4).

layer ℓ	consumes level	sourced	phase-A entry
$0 \dots d-4$	$1 \dots d-3$	stored chain	Dense
$d-3$	$d-2$	JIT: one <i>T4</i> gather/value	Dense (round 1 fused)
$d-2$	$d-1$	implicit	Pair3Bits over <code>(te,to)</code>
$d-1$	d (leaves)	implicit	Leaf3Bits over τ

The stored chain is built per tree directly at level $d-3$ — each value the product of two paired *T4* gathers — so levels $d-2$, $d-1$, d never exist at build time, and the chain then folds upward ($\sum_{\ell \leq d-3} 2^\ell \approx 2^{d-2}$ values per tree, a quarter of the leaf count; hence the name). Reading the bits word-wise during these builds costs 4 `u64` loads per 64 values, $\approx 64\times$ less read traffic than per-bit gathers over the column-lane store. At depth 4 exactly (and for every depth under the `F2Z_LUT3=0` opt-out) the bottom two layers run the shallower **Pair2Bits** / **Leaf2Bits** variants; depth 3 keeps only the leaf layer bit-driven (**LeafBits**) with the chain built from level-1 halves by a zero-multiply case copy (`build_column_layer1_halves`); depth < 3 falls back to the eager prover (the LUT rounds need $k \geq 2$).

The L/8 schedule (F2_FOREST_SCHEDULE=18; depth ≥ 5).

layer ℓ	consumes level	sourced	phase-A entry
$0 \dots d-5$	$1 \dots d-4$	stored chain	Dense
$d-4$	$d-3$	JIT: two <i>T4</i> gathers + 1 mul /value	Dense (round 1 fused)
$d-3$	$d-2$	implicit	T4Bits over <i>T4</i>
$d-2$	$d-1$	implicit	Pair3Bits over <code>(te,to)</code>
$d-1$	d	implicit	Leaf3Bits over τ

Every stage's resident set is $\approx 2^{d-3}$ values per tree — an eighth of the leaf count — at a measured +5–13% prove cost at the 2-column model shapes; the schedule exists for memory-bound runs and is *required* at $n \geq 31$ on a 16 GB box (§6).

5.3 Inside the case-LUT rounds

The bit-driven layers are where “lazy” earns its keep: their input levels are never materialised because the first one, two, or three sumcheck rounds of the layer can be computed *directly* from (`lbits`, `rbits`) and per-position case tables. All of the following are exact char-2 identities — the transcripts they produce are bit-for-bit those of dense buffers.

The leaf round (LeafBits family). For the leaf layer, entry i of the phase-A pair is *defined* as $L[i] = 1 + \text{lbits}[i] \cdot \tau_L[i]$ (and R from rbits, τ_R). Expanding round 1’s per-slot contribution $w_b L(c)R(c)$ over the four selecting bits, the 1’s cancel (char 2) into subset sums of eight tree-shared $w\tau$ -product tables (**build_leaf_tables**: the 4-case singles/pair tables t_{a_0}, t_{a_1} and a $\Delta L \Delta R$ table), so a *group*’s round-1 message costs **zero multiplications**: per slot, one unconditional indexed load and add per accumulator (3 loads + 3 adds), branchless on the bit nibble. The tables themselves cost 8 multiplies + ≈ 20 adds per slot to build — once, amortised over all 2^s trees. The $\Delta\Delta$ table has two interchangeable forms, chosen by footprint (**F2Z_LEAF_A2_FACTORED**, auto at $2^{k-1} \geq 2^{15}$): a 16-case precombined table (one load per slot; wins while L2-resident) or the four raw cross products with branchless masked adds ($4\times$ smaller; at $n = 30$ the leaf round drops $2.1\times$, $307 \rightarrow 147$ ms, prove -6.5%).

Folding without materialising. Round 1’s fold must produce round 2’s buffers. For a bit-affine leaf side this is again a 4-case table: the folded entry is $1 + m_0(1+\rho_1)\tau_0 + m_1\rho_1\tau_1$, precombined per position over the adjacent bit pair (m_0, m_1) (**build_leaf_fold_tables**), so the fold is one indexed load per entry. The deeper variants iterate the idea instead of executing it:

- **Leaf2Bits** ($k \geq 3$): round 1’s fold *keeps the bits* and stashes its fold tables as round 2’s *value* tables — which are structurally a (**te**, **to**) pair, so round 2 runs the 16-case product-layer round below, and only round 2’s fold materialises dense buffers (2^{k-2} per side);
- **Leaf3Bits** ($k \geq 4$, the default): one round deeper still; round 3 reads the stashed 16-case fold tables *inline* (one aligned byte load per side per slot — the measured lesson being that multiplies are free and only skipped bytes pay), and dense buffers appear only at round 3’s fold (2^{k-3} per side).

The product-layer round (Pair2Bits/Pair3Bits). One level up, entry y of the pair is the 2-bit select $\text{te}[4y + (m_{lo}|m_{hi} \ll 1)]$ (and **to** at the offset position), over the *same* leaf-bit halves. The default round body is the *factored* form (**F2Z_PAIR2_FACTORED**): only the suffix-weighted table $\text{wte}[i] = V_1[i \gg 3] \cdot \text{te}[i]$ is built, and each slot does three wide multiplies against **wte**/**to** entries — trading one multiply for a $4\times$ smaller, L2-resident table set (the precombined 16-case alternative carries $64 \cdot 2^k$ entries, 16 MiB at deployed shapes, and its four gather streams’ misses dominate the round). **Pair3Bits** again stashes round 1’s 16-case fold tables as round 2’s inline value tables and materialises at round 2’s fold.

The 4-leaf round (T4Bits, L/8 only). One further level up, entry y is a single 16-case select from the shared $T4$ — one gather per value, values read inline in round 1, dense buffers at its fold. The layer’s input level is never stored *nor* regenerated.

Prefetch. The LUT gathers are data-dependent (committed bits pick the line), which defeats the hardware prefetcher, but the indices are cheaply recomputable ahead: both the stash-gather rounds (**F2Z_LUT_PRFM**) and the $T4$ build/JIT sites (**F2Z_T4_PRFM**) issue **prfm pldl1keep** 16 slots ahead, size-gated to the DRAM-scale shapes (they *lose* at L2-resident sizes; measured -4.8% and -5.3% prove at $n = 30$ respectively). Semantically inert.

5.4 Why byte-identity holds, and how it is pinned

Every lazy device above is a re-*association* of the same field arithmetic: case tables precombine subexpressions that the dense path would compute per entry (field addition is exact and

commutative); char-2 cancellations ($1 + 1 = 0$, $v_1 - v_0$ ’s ones cancelling in folds) are identities; wide-vs-narrow accumulation reduces to identical values because the reduction is $\mathbb{F}_2$ -linear; fusion only reorders physical passes. Consequently the lazy prover does not merely “verify the same” — it emits the *same transcript*. `lazy_matches_eager` pins roots, every layer’s sumcheck messages, pairs, exit claim, and the post-run transcript state against the eager prover across depths 3–9 (covering every fallback edge and both the scalar and word-wise $T4$ paths), $W \in \{1, 4\}$, and *both* schedules; `lazy_multi_matches_eager` does the same for the batched forest (§8). This is why the knob inventory of Table 1 is safe: every knob changes scheduling, not semantics.

Knob	Default	Effect (always byte-identical)
<code>F2_FOREST_SCHEDULE=18</code>	off (L/4)	L/8 memory schedule (§5.2)
<code>F2Z_LUT3=0</code>	on	shallower <code>Pair2/Leaf2</code> LUT prefixes
<code>F2Z_EQF_FUSE=0</code>	on	eager two-pass fold (no pass fusion)
<code>F2Z_JIT_R1=0</code>	on	unfused JIT round-1 message pass
<code>F2Z_LEAF_A2_FACTORED=0/1</code>	auto ($\geq 2^{15}$)	$\Delta\Delta$ table form, leaf round
<code>F2Z_PAIR2_FACTORED=0</code>	on	precombined 16-case pair tables
<code>F2Z_LUT_PRFM=0/1</code>	auto ($\geq 2^{14}$)	stash-gather software prefetch
<code>F2Z_T4_PRFM=0/1</code>	auto (≥ 16 MiB)	$T4$ build/JIT software prefetch
<code>F2Z_EQF_NOKERNEL=1</code>	off	bypass fused NEON kernels (diagnostic)

Table 1: The forest’s scheduling knobs. Each is read once per prove/process; none changes the proof bytes.

6 Memory and cost

Memory model. With 16-byte $\mathbb{K}$ -elements and $W = 1$ (so $2^{d+s} = 2^n$ leaves), the resident forest set is capped by the largest stage of the ladder:

$$\text{L/4: } \approx 2^{n-2} \cdot 16\text{ B} = 2^n \cdot 4\text{ B}, \quad \text{L/8: } \approx 2^{n-3} \cdot 16\text{ B} = 2^n \cdot 2\text{ B},$$

(the stored chain and the JIT level are the same size and not simultaneously live: the chain is consumed front-first, `mem:take` per layer, before the JIT layer runs). On top sit the commit hint ($\approx 2^n \cdot 0.7\text{ B}$: packed rows/columns, codeword, Merkle tree), the transposed bit halves ($2^n/8\text{ B}$), and the shared tables ($O(2^d)$ per set; the largest, $T4$, is $16 \cdot 2^{d-2}$ entries — 8 MiB at $d = 17$). The model matches the measured peaks below to within $\approx 5\%$; it is why the 16 GB ceiling sits at $n = 32$ (under 18) and why $n \geq 31$ requires 18 at all.

Measured (Apple M4 16 GB, LTO + unchecked, fusion defaults, rate-1/2 $k=4$ configs, medians; 2026-07-18). The `forest+presum` column is the phase this note describes (the α -power tables, the forest proper, and the pre-sumcheck of §7); its share of prove rises from 81 % to $\approx 96\%$.

n	(t, s)	forest+presum	lig. open	prove	verify	peak	sched
16	(10, 6)	3.25 ms	0.72 ms	4.03 ms	1.21 ms	0.69 MB	L/4
20	(13, 7)	9.13 ms	1.62 ms	10.9 ms	1.83 ms	7.7 MB	L/4
24	(15, 9)	42.5 ms	21.5 ms	62.1 ms	2.48 ms	85.0 MB	L/4
26	(16, 10)	131 ms	25.3 ms	159 ms	3.14 ms	325 MB	L/4
28	(17, 11)	530 ms	53.6 ms	579 ms	4.41 ms	1.27 GB	L/4
30 [†]	(18, 12)	2.98 s	139 ms	3.15 s	5.55 ms	4.95 GB	L/4
32 [†]	(19, 13)	~96 %	~4 %	29.8 s	11.8 ms	11.5 GB	18

([†]: the older $k=6$ generation, pending a memory-fresh re-measure; full tables and the measurement protocol in `README.md`.) Proof bytes stay forest-light: forest-side 7.7 KiB ($n=16$) $\rightarrow$ 50.7 KiB ($n=28$) $\rightarrow$ 151.8 KiB ($n=32$), of which the sent folds are $16 \cdot 2^s$ B (§2); the Ligerito blob dominates the stream throughout. Threading: the build is parallel across trees, round bodies across groups (with a minimum batch so tiny late rounds amortise dispatch), phase B and the upper layers effectively serial; the MT/ST prove ratio grows from $\approx 1.6\times$ at $n = 22$ to $\approx 3.6\times$ at $n = 28$. All deployed shapes run a *single* chunk (with $q = 2^{100}-15$, $c_w = 126 - t \geq 100$ for $t \leq 26$); chunking multiplies the forest count only at $t + W > 27$.

7 The exit claim and its discharge

The forest ends holding $e_d \approx \tilde{\Lambda}(z_x, z_c)$ — still entangled with the α -power weights. Two further steps (`prove_int_eval_merged_common`) turn it into something the opener understands; they are the “presum” half of the `forest+presum` phase.

From the exit claim to a bit-MLE claim. By (1) and multilinearity (the all-ones table extends to 1):

$$e_d - 1 = \sum_{i,c} \text{eq}(i, z_x) \text{eq}(c, z_c) M[c][i] \tau(i) = \sum_{i \in \{0,1\}^d} q_{\text{rowbit}}[i] \cdot m[i],$$

with $q_{\text{rowbit}}[i] = \text{eq}(i, z_x) \tau(i)$ (`row_bit_weights` at $\rho = z_x$; its $2^t \approx q_{\text{bits}}$ -bit exponentiations go through the same fixed-base comb) and $m[i] = \sum_c \text{eq}(c, z_c) M[c][i]$ the z_c -combined row (`xi_combined_rows`). A d -round *degree-2* sumcheck over this product (`MultiDegreeSumcheck`, one group) binds $i \rightarrow r^*$; dividing the final expected evaluation by the public $\widetilde{q_{\text{rowbit}}}(r^*)$ leaves $\mu = \widetilde{M}(r^*, z_c)$ — a pure bit-MLE evaluation at $(r^* \| z_c)$, every α -dependent factor stripped to the verifier’s side. From here the ring switch sends the 128 partial evaluations s_v , the L chunks’ residuals are η -combined, and one recursive Ligerito call certifies the batch against the commitment — verifier note §6, out of scope here.

The chunked flow. `prove_mle_eval_mod_q_ligerito` runs the full forest + presum *per chunk*, sequentially on one transcript (chunk l ’s α -power tables are built and dropped inside its run), then does all ring-switch messages, then the single Ligerito call. Per chunk the proof stream carries: the `MergedForestProof`, the folds $u^{(l)}$, the pre-sumcheck, and s_v (`proof_codec.rs`).

8 Variants: the batched forest and the RLC families

Multi-claim batching (`prove_merged_forest_lazy_multi`). N same-shape claims — each 2^s trees over its own bits and τ chains — run as *one* forest of $N \cdot 2^s$ trees, tree index $(\nu \ll s) | c$ with

the claim index high, sharing every layer’s rounds and messages. Per-claim table selection rides the driver’s `tau_set` index (the kernels are untouched); table sets are deduplicated by τ -chain identity, so same-point claims share one set. N must be a power of two; callers pad with zero-weight dummies, whose τ chains are all 1 and whose leaves are therefore identically 1 regardless of the bits they carry. Both schedules honoured; pinned byte-identical to the eager prover over the concatenated leaf tables, dummy arm included.

RLC claim families (experimental). The mod- q RLC families (`docs/rlc-family-note/`) replace the bit-affine leaf with a 2^j -case select on j committed column bits: $\Lambda[c][i] = \text{case_pow}[i][m(i, c)]$, case $0 = \alpha^0 = 1$. For $j = 2$ (`prove_merged_forest_lazy_rlc2`) this is a pure *rewiring* of the machinery above, one level shifted: the leaf layer is structurally a product layer (`Pair3Bits` over `te[4y+m] = case_pow[y][m]`), level $d-1$ is a `T4Bits` round over `te · to` products, level $d-2$ is Dense JIT from paired gathers, and the chain tops at $d-3$ — the L/4 shape with untouched kernels, hence byte-identical to the eager forest over the same leaves (pinned). For $j = 3, 4$ (`prove_merged_forest_lazy_rlc_general`) no 2^j -case leaf kernels exist yet, so the bottom layers run Dense with JIT-generated buffers gathered from a shared 2^{2j} -case top-pair product table $T2$; dedicated 8/16-case leaf-round kernels are the stated open lever. (A Dense-JIT arm for the *bit-affine* $j \geq 3$ forests was measured slower than the eager path and closed.)

9 What to trust, and what this note is not

The prover described here is exercised by: the both-schedule byte-identity pins (`lazy_matches_eager`, `lazy_multi_matches_eager` — roots, every message, exit claim, transcript state, across the depth-regime and W matrix), the GKR roundtrip and tamper rejections (`merged_forest_roundtrips`), the mod- q end-to-end roundtrips with range/generator/tamper arms, and the NEON/scalar field-pipeline equivalence (`neon_mul_matches_scalar_pipeline`). Soundness of what the forest *binds* — the chain from the absorbed roots through the layer sumchecks to the exit claim, the exponent injectivity, and the oracle handoff — is argued in the verifier note and `docs/DESIGN.md`; this note’s claim is narrower and mechanical: the lazy, merged, fused prover computes *the same protocol* as the eager reference, at $O(\ell + s)$ payload per layer, $\approx 2^n \cdot 4\text{ B}$ (or $2^n \cdot 2\text{ B}$) of working set, and 81–96 % of an F2Z prove. The standard caveats carry over unchanged: flock’s Merkle domain separation is a flagged upstream item, the ad-hoc Ligerito config regime ($m < 22$) is unaudited, and the scheme is not zero-knowledge.

References

- [1] S. Goldwasser, Y. T. Kalai, G. N. Rothblum. *Delegating computation: interactive proofs for muggles*. J. ACM 62(4), 2015 (STOC 2008).
- [2] J. Thaler. *Time-optimal interactive proofs for circuit evaluation*. CRYPTO 2013. (The product-tree GKR specialisation the forest layers instantiate.)
- [3] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, D. Song. *Libra: succinct zero-knowledge proofs with optimal prover computation*. CRYPTO 2019. (The eq-factored linear-time GKR prover lineage.)
- [4] A. Gruen. *Some improvements for the PIOP for ZeroCheck*. IACR ePrint 2024/108. (Tail-only round messages via the claimed-sum reconstruction.)

- [5] L. Soukhanov. *The characteristic-2 field-switching note*, §9. (The integer-MLE-over- $\mathbb{F}_2$ construction F2Z realises; internal.)